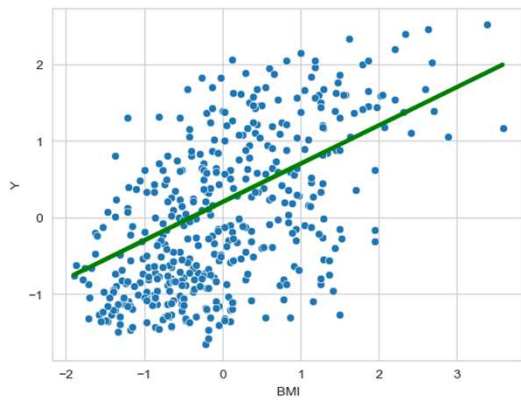
Neural networks

Ralf Gabriels

Machine learning methods in biomedical data
Academic year 2025-2026

A quick recap on linear regression



$$f(x) = 0.5 \cdot x + 0.2 = ax + b$$

a and *b* are **model parameters**

a is the **slope** to the line (direction)

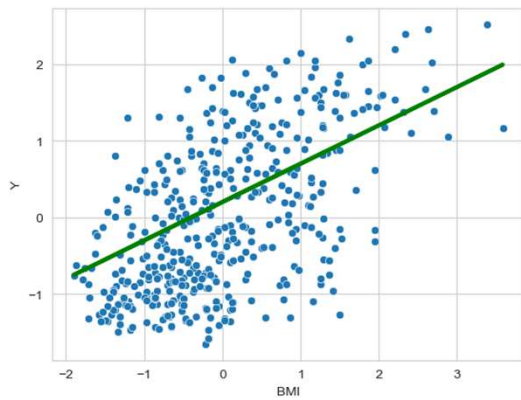
b is the **intercept** or bias (position)

$$f(x, \theta) = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots + \theta_m x_m$$

2

In a **simple linear regression** model, the prediction is a weighted combination of the input plus an intercept. The parameters determine the direction and position of the fitted line.

A quick recap on linear regression



$$f(x) = 0.5 \cdot x + 0.2 = ax + b$$

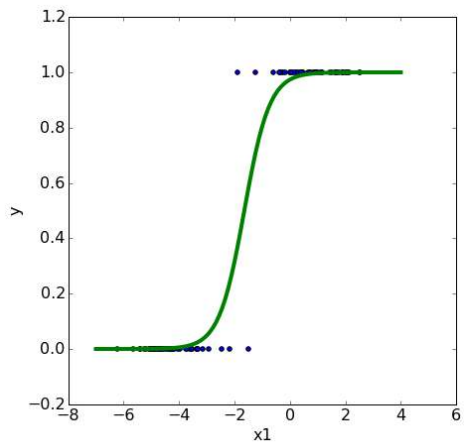
More inputs,
More parameters

$$f(x, \theta) = \theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots + \theta_m x_m$$

3

When more input **features** are added, the model gains more **parameters**, because each feature needs its own weight.

A quick recap on *logistic* regression



$$f(x, \theta) = g(\theta_0 + \theta_1 x_1)$$

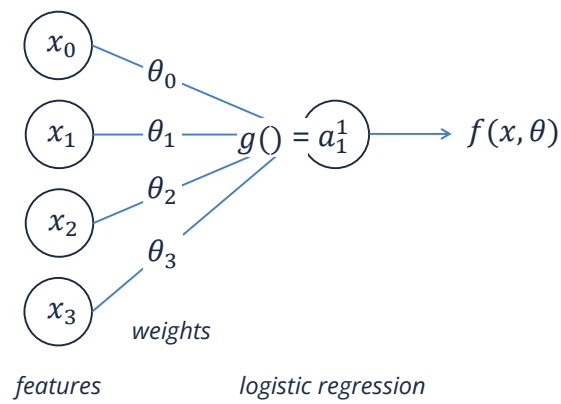
More inputs,
More parameters

$$f(x, \theta) = g(\theta_0 x_0 + \theta_1 x_1 + \theta_2 x_2 + \ldots + \theta_m x_m)$$

4

For classification, **logistic regression** replaces the direct linear output by a **non-linear transformation**, typically a sigmoid. The model still computes a weighted sum of the inputs, but then maps that score to a **probability-like output** between 0 and 1. This makes logistic regression suitable for **binary classification**.

Logistic regression could be visualized as a network of information, flowing from features and weights to the output

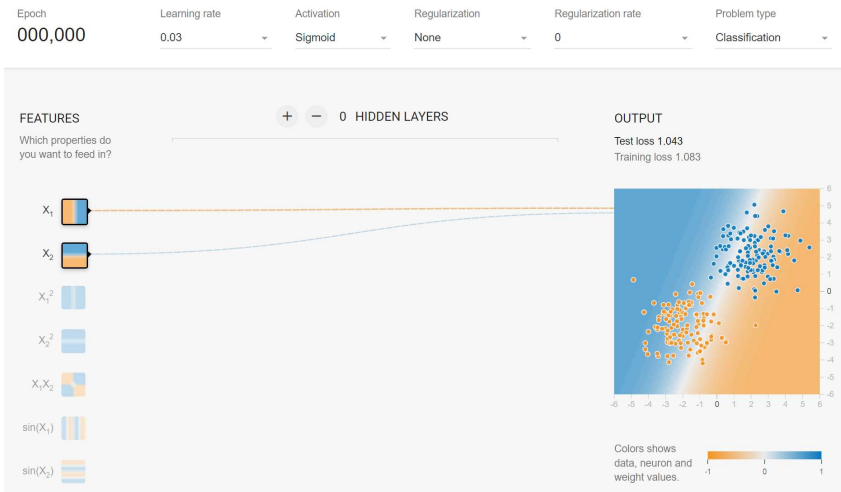


5

Logistic regression can be viewed as a **very small network**: input features are multiplied by weights, summed together with a bias, and passed through an activation function to produce the output.

This viewpoint is useful because it shows that neural networks are not completely new models, but rather structured combinations of simpler regression-like units.

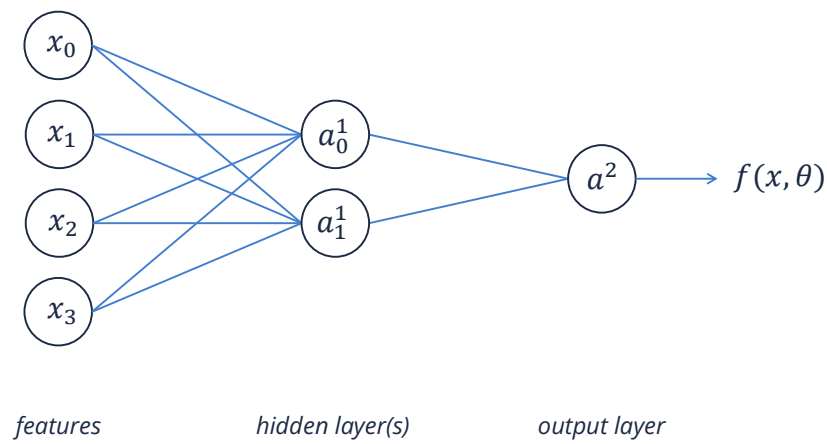
Logistic regression could be visualized as a network of information, flowing from features and weights to the output



https://playground.tensorflow.org

A neural network is simply a layered stack of regression models

The features for each node (a regression model) are the output from the nodes in the previous layer

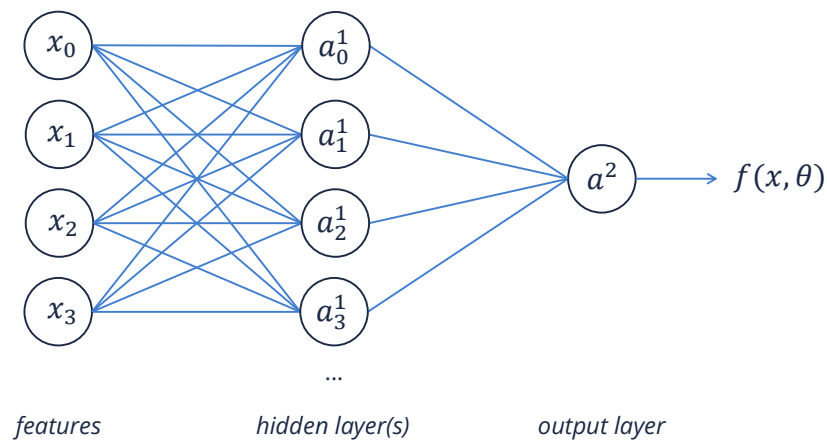


7

A **neural network** is a **layered stack of such units**. The first layer receives the original input features. Each **node** in a **hidden layer** computes a transformed version of the previous layer's outputs. The next layer then uses those transformed outputs as its own inputs. In this way, the network builds increasingly abstract internal representations of the data.

A neural network is simply a layered stack of regression models

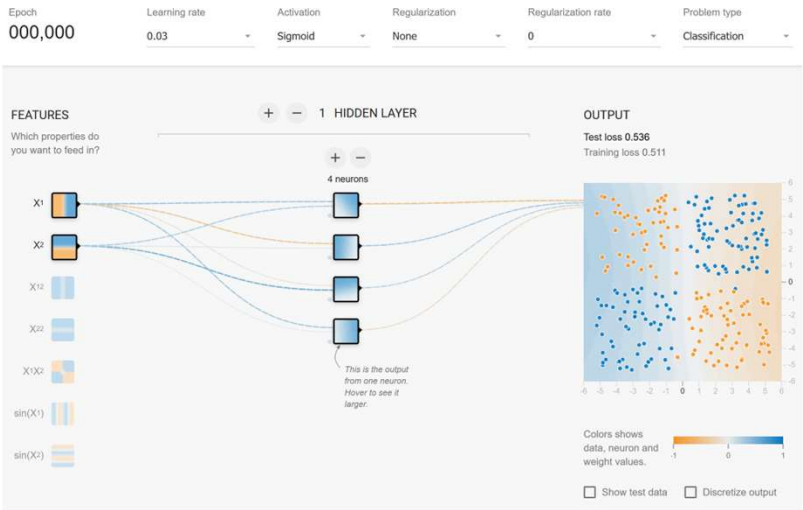
The features for each node (a regression model) are the output from the nodes in the previous layer



8

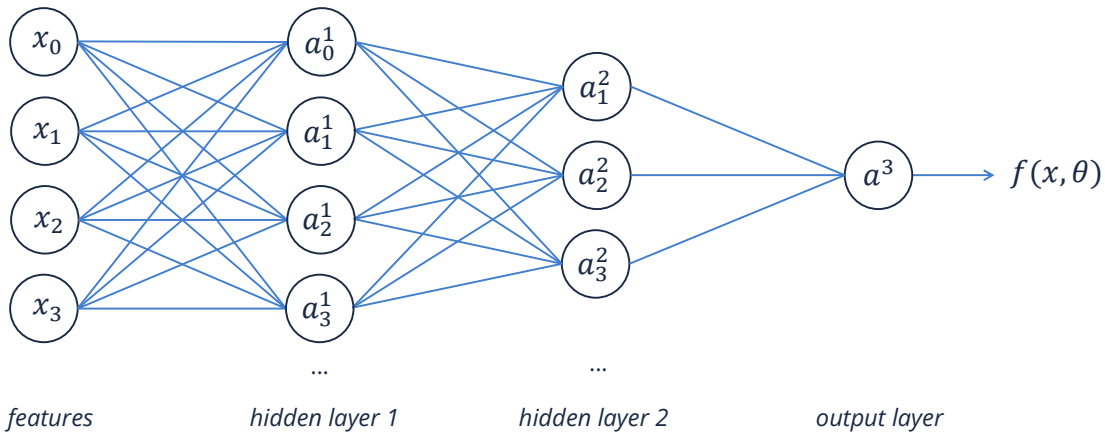
A shallow neural network contains one hidden layer. A deeper neural network contains multiple hidden layers. Adding layers allows the model to **capture more complex patterns** than a single logistic regression can represent.

A neural network is simply layered stacks of logistic regressions



https://playground.tensorflow.org

A deep neural network adds many layers to capture complex patterns

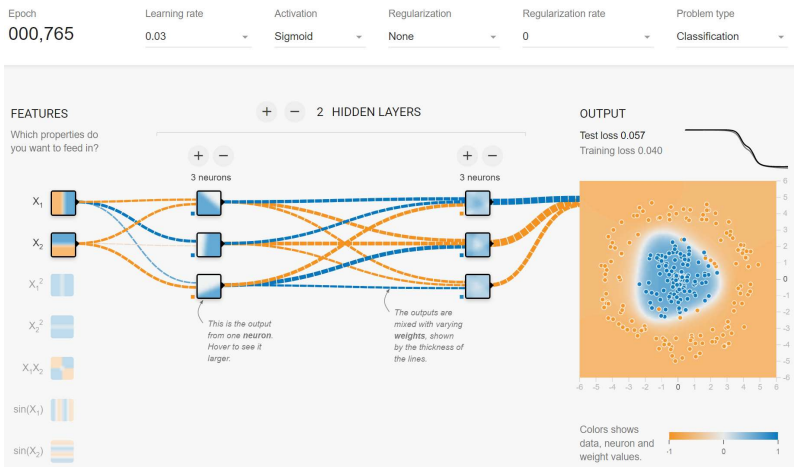


10

Deeper networks can learn more complex relationships because each layer composes transformations from the previous layer. Earlier layers may capture simple patterns, while later layers combine these into more sophisticated structures.

However, this added flexibility comes at a cost: deeper models usually require more data and more careful training.

A deep neural network adds many layers to capture complex patterns



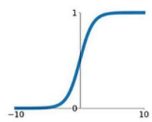
https://playground.tensorflow.org

Activation functions can be added on the output of each node

- ➡️ Capture non-linear relationships
- ➡️ Avoid exploding or diminishing outputs in deep networks

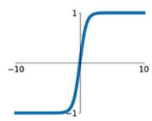
Sigmoid

$\sigma(x) = \frac{1}{1+e^{-x}}$



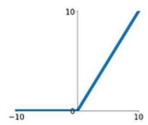
tanh

$\tanh(x)$



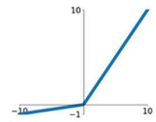
ReLU

$\max(0, x)$



Leaky ReLU

$\max(0.1x, x)$

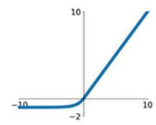


Maxout

$\max(w_1^T x + b_1, w_2^T x + b_2)$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



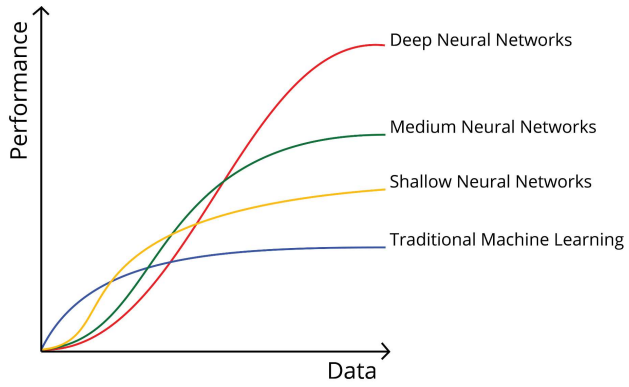
12

Activation functions are applied to the output of individual nodes. Their role is crucial because they introduce **non-linearity**. Without them, stacking many layers would still behave like a single linear transformation, and the network would not gain much expressive power.

Do we need to go deeper?

Deeper networks...

- ➡️ **... can learn more complex patterns**
- ➡️ **... require more data to be trained**

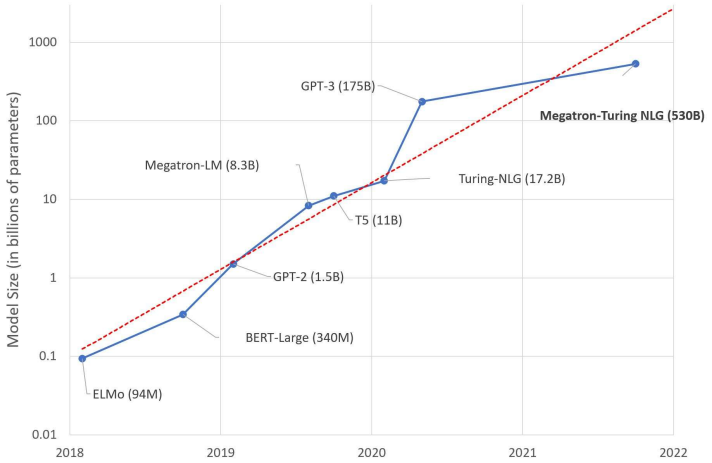


13

Do we always need to “go deeper”? The answer is nuanced. Deeper networks can learn more complex patterns, but they also require more training data and computational resources. For smaller datasets or simpler problems, a shallow network or even a traditional machine learning model may be more appropriate.

For complex tasks, deeper seems better...

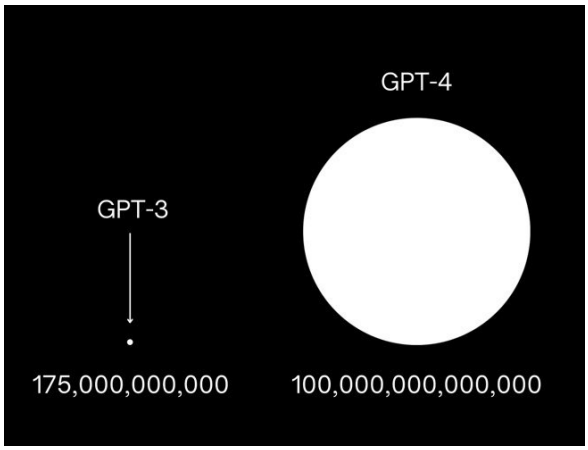
However, more complicated architecture setups are required to make the models learn well.



https://huggingface.co/blog/large-language-models

For complex tasks, deeper seems better...

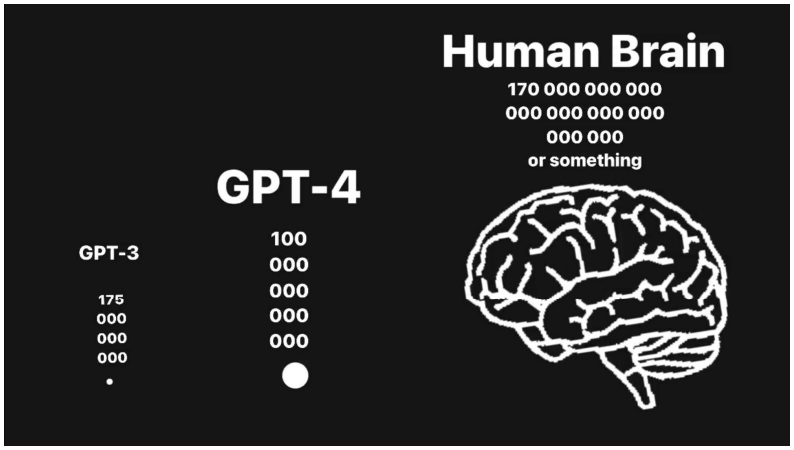
However, more complicated architecture setups are required to make the models learn well.



https://openai.com/product/gpt-4

For complex tasks, deeper seems better...

However, more complicated architecture setups are required to make the models learn well.



https://openai.com/product/gpt-4

The *loss function* should be minimized to find the optimal model

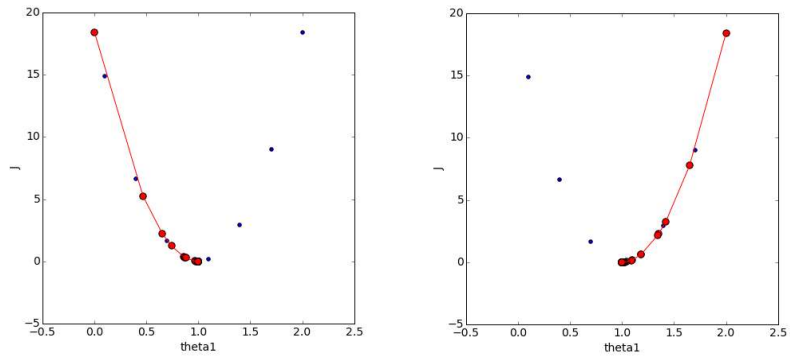
regression	➡️	MSE
binary classification	➡️	binary cross entropy
multi-class classification	➡️	categorical cross entropy
…	➡️	…

17

As in other machine learning models, neural networks are trained by **minimizing a loss function**. The choice of loss depends on the task. **Mean squared error** is typical for regression, **binary cross-entropy** for binary classification, and **categorical cross-entropy** for multi-class classification. The objective is always the same: find parameter values that make the model's predictions fit the targets as well as possible.

Neural nets also rely on *gradient descent* to minimize the loss

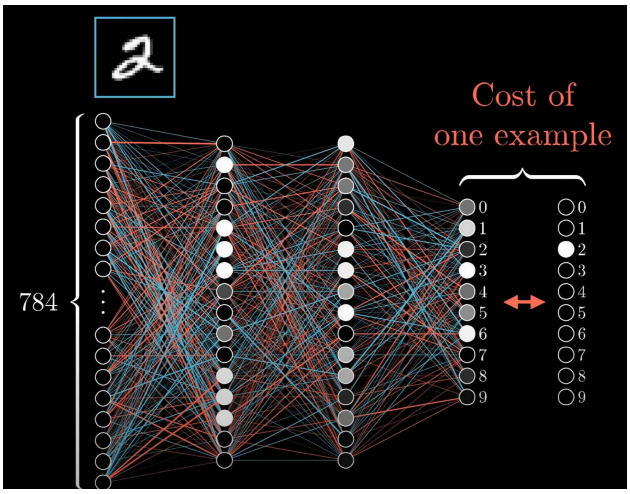
Iteratively adjusting the weights and biases in the direction that reduces the loss



18

Neural networks are optimized using **gradient descent**. Training consists of iteratively **adjusting weights and biases** in the direction that reduces the loss. This is the same core optimization idea already introduced for simpler models, but now applied to a **much larger number of parameters**.

The big trick is to use *backpropagation* to efficiently calculate gradients for each parameter across many layers

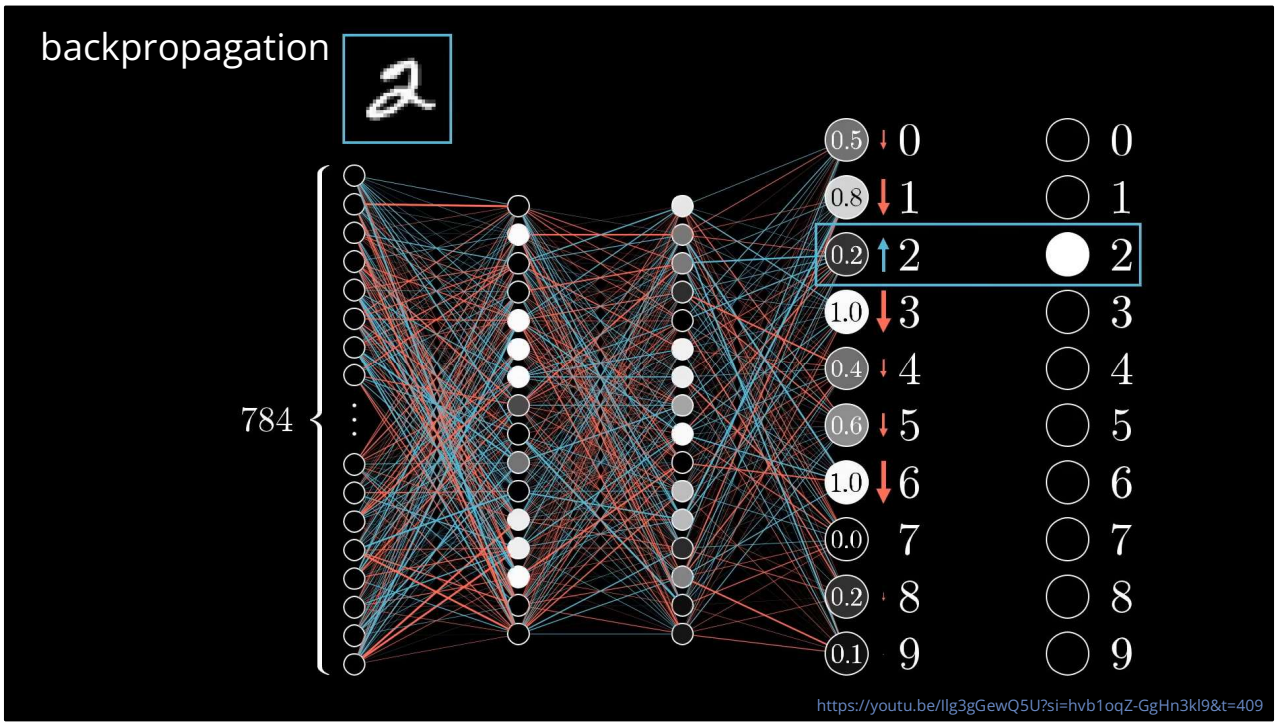


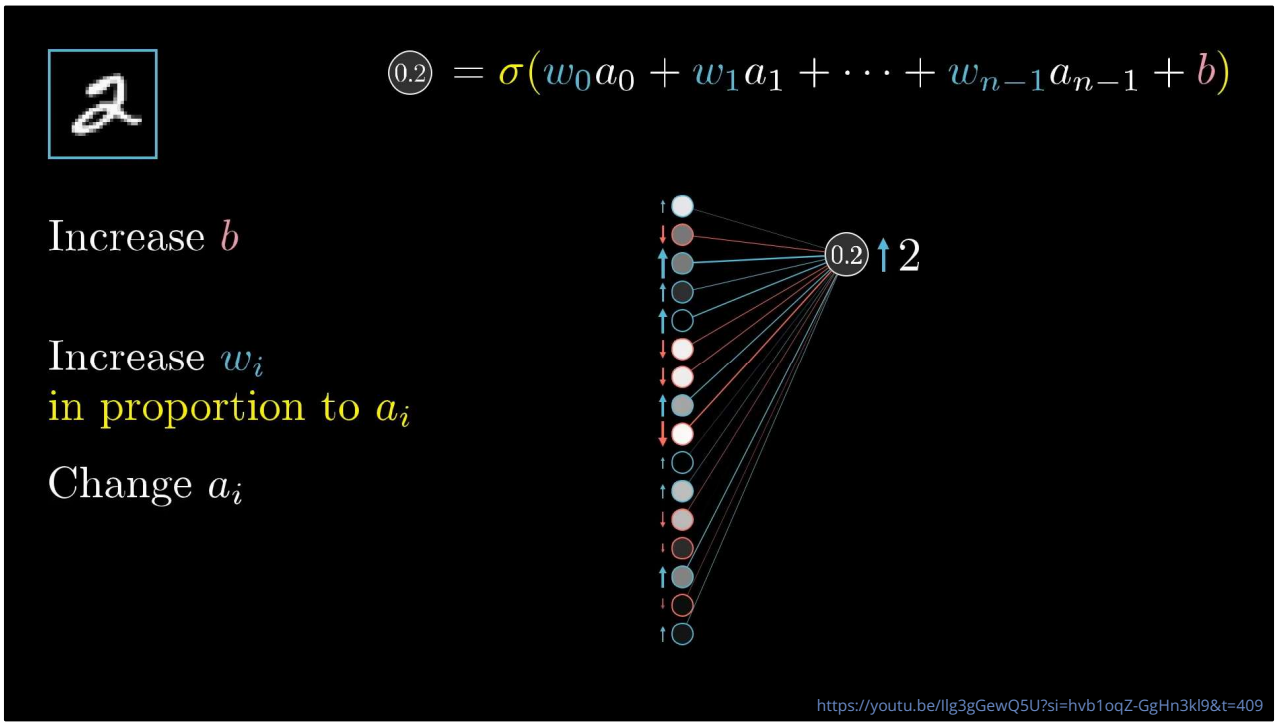
https://www.youtube.com/watch?v=Ilg3gGewQ5U&t=548s

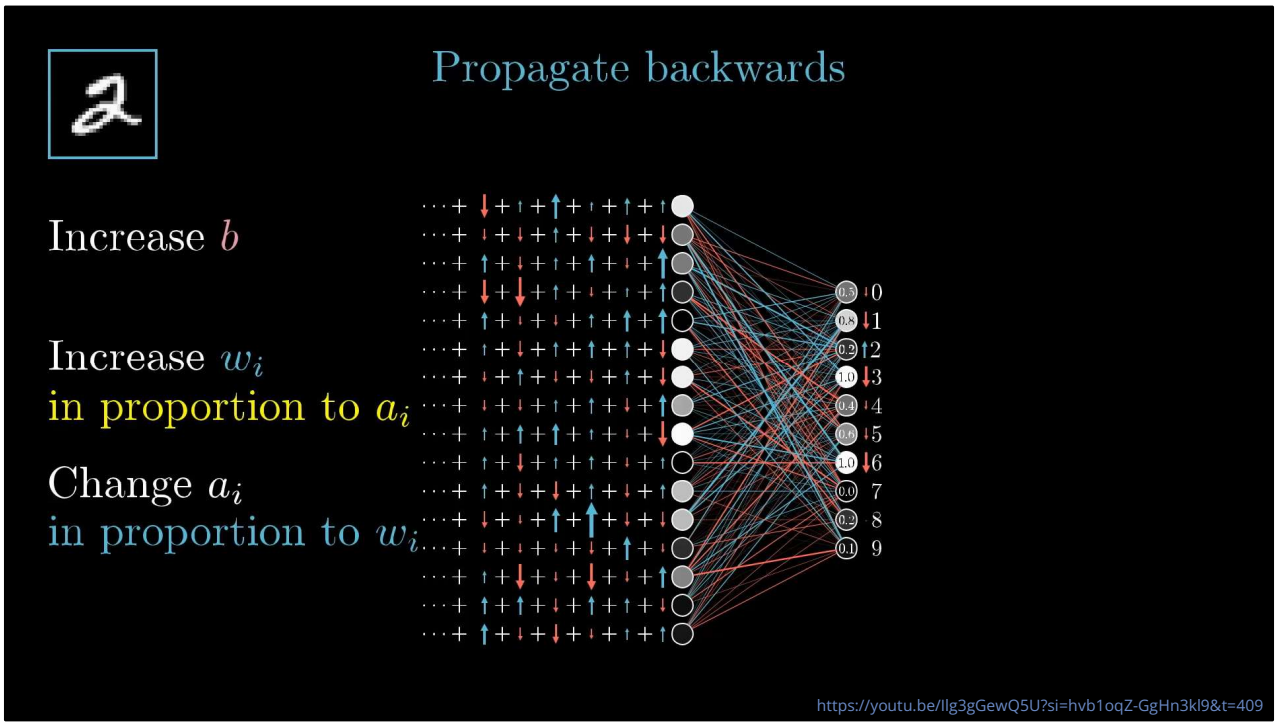
19

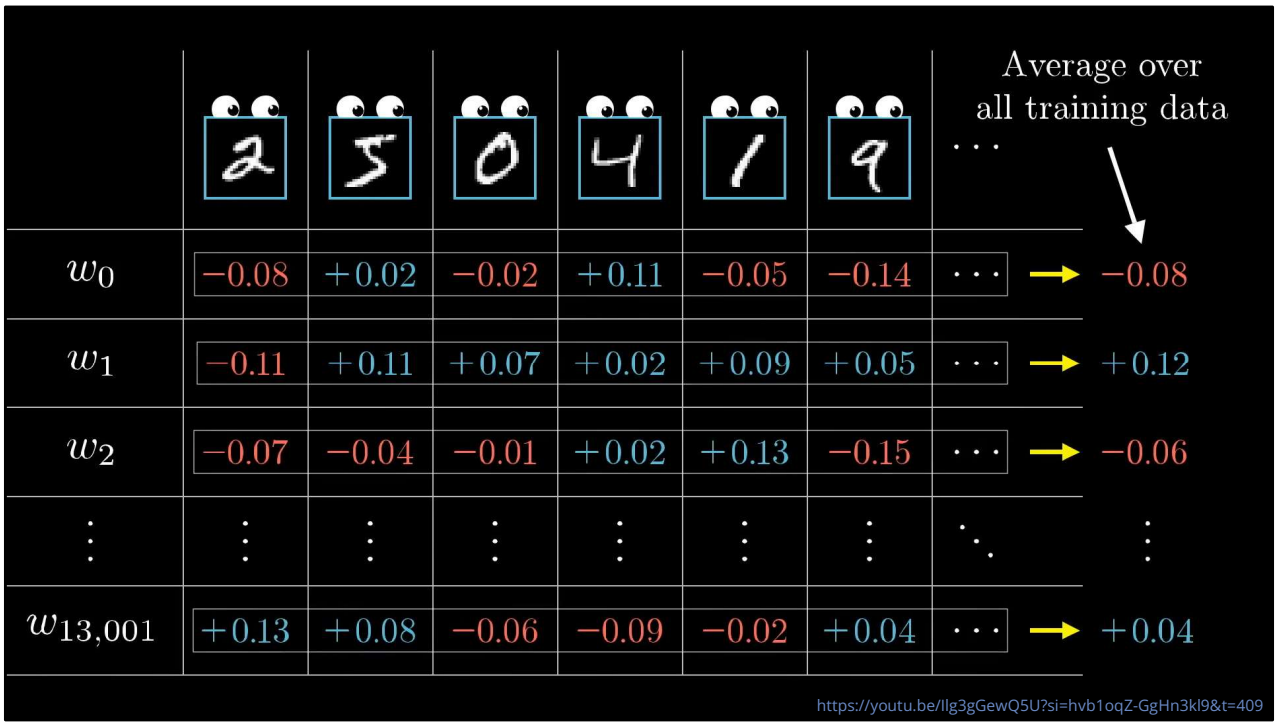
The key technical idea that makes neural networks trainable is **backpropagation**. In a network with many layers and many parameters, directly computing the influence of every parameter on the loss would be inefficient. Backpropagation solves this by propagating error information backward through the network and efficiently calculating gradients for all parameters.

Conceptually, the forward pass computes activations and predictions. The backward pass then determines how each weight and bias contributed to the final error and how it should change to reduce that error. This lets deep models with many layers be trained in practice.

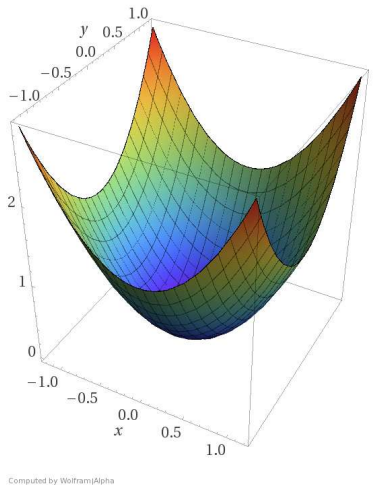

backpropagation
784
0.5
0.8
0.2
1.0
0.4
0.6
1.0
0.0
0.2
0.1
0
1
2
3
4
5
6
7
8
9
0
1
2
3
4
5
6
7
8
9
https://youtu.be/Ilg3gGewQ5U?si=hvb1oqZ-GgHn3kl9&t=409


$0.2 = \sigma(w_0 a_0 + w_1 a_1 + \cdots + w_{n-1} a_{n-1} + b)$
Increase b
Increase w_i
in proportion to a_i
Change a_i
0.2
2
https://youtu.be/Ilg3gGewQ5U?si=hvb1oqZ-GgHn3kl9&t=409

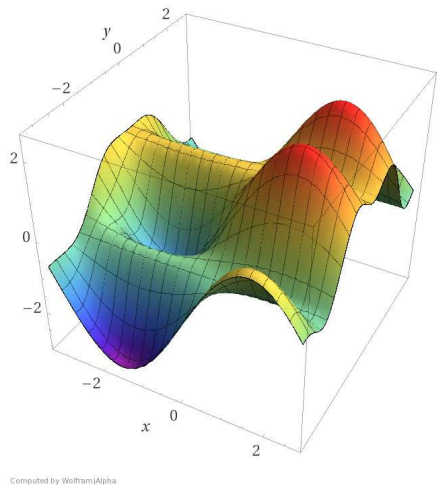

Propagate backwards
Increase b
Increase w_i
in proportion to a_i
Change a_i
in proportion to w_i
0.5 0
0.8 1
0.2 2
1.0 3
0.4 4
0.6 5
1.0 6
0.0 7
0.2 8
0.1 9
https://youtu.be/Ilg3gGewQ5U?si=hvb1oqZ-GgHn3kl9&t=409


Average over all training data
w_0 −0.08 +0.02 −0.02 +0.11 −0.05 −0.14 ··· → −0.08
w_1 −0.11 +0.11 +0.07 +0.02 +0.09 +0.05 ··· → +0.12
w_2 −0.07 −0.04 −0.01 +0.02 +0.13 −0.15 ··· → −0.06
$w_{13,001}$ +0.13 +0.08 −0.06 −0.09 −0.02 +0.04 ··· → +0.04
https://youtu.be/Ilg3gGewQ5U?si=hvb1oqZ-GgHn3kl9&t=409

The loss surface can be highly complex, with many local minima



logistic regression

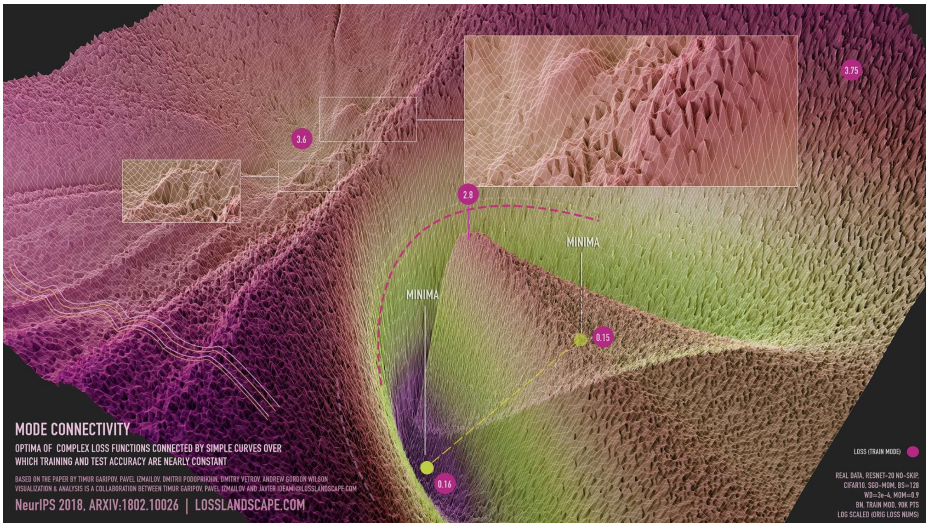


(small) neural network

24

For simple models such as logistic regression, the **loss surface** is often relatively smooth and easy to optimize. For neural networks, especially deep ones, the loss surface becomes much more complex, with many **local minima** and irregular regions. This makes optimization more difficult and partly explains why training deep networks requires careful design choices.

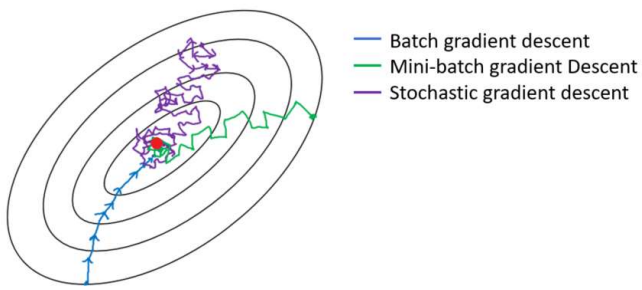
The loss surface can be highly complex, with many local minima



deep neural network

Mini batch is the goldilocks method of gradient descent

	Batch gradient descent	Stochastic gradient descent	Mini-batch gradient descent
Method	Use entire dataset	Use one sample at a time	Use a small subset of the data
Update stability	Every step is accurate	Steps are very noisy	Steps are somewhat noisy
Computationally expensive?	Expensive (large computation for each update)	Very expensive (no vector parallelization)	Moderate cost
Update speed	Slow to update	Very fast per update	Moderate speed



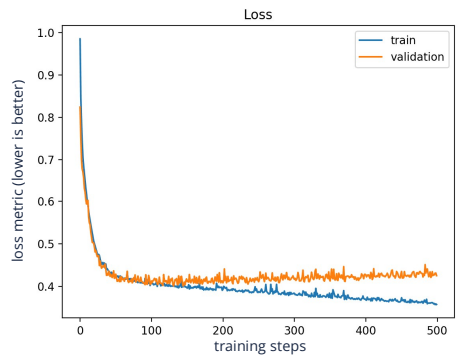
26

In **batch gradient descent**, each update uses the entire dataset. This makes updates stable but computationally expensive. In **stochastic gradient descent**, updates are made using one sample at a time, which is fast per step but very noisy. **Mini-batch gradient descent** uses a small subset of the data for each update and offers a practical compromise between stability and efficiency.

Neural networks are trained over multiple *epochs*

General training loop

- For each epoch
  - For each mini-batch (one step)
    - Forward pass
      make predictions
    - Calculate loss
      compare with targets
    - Calculate gradients (aka backward pass)
      how should parameters be updated?
    - Update model parameters
  - Forward pass on validation data
    - Did model improve? Continue.
    - Did it stop improving? Stop training.



When should you stop fitting this model?

Figure from https://machinelearningmastery.com/learning-curves-for-diagnosing-machine-learning-model-performance/

27

Neural networks are trained over multiple **epochs**, where one epoch means one full pass through the training data. Within each epoch, the model processes many mini-batches. For each mini-batch, the network performs a **forward pass**, computes the loss, performs a **backward pass** to calculate gradients, and then updates the parameters.

After training updates, the model is also evaluated on **validation data**. If validation performance improves, training can continue. If validation performance stops improving, training should be stopped. This is an example of **early stopping**, which helps prevent overfitting. The learning-curve figure shows the typical situation where training loss continues to decrease while validation loss eventually stops improving or begins to worsen.

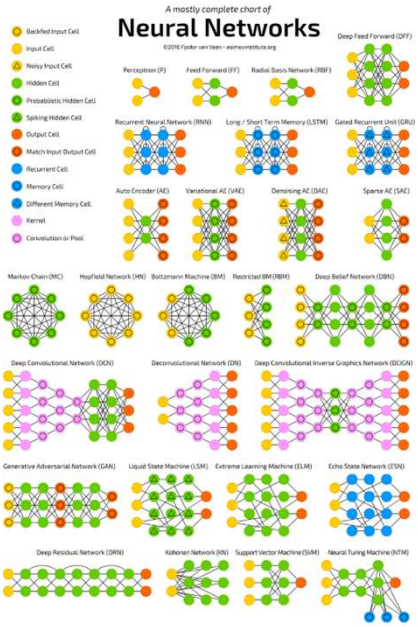
Neural networks are highly flexible

modules == building block == transformation == function

a module receives as input either data x or another module's output.

a module returns an output based on its activation function

a module may or may not have trainable model parameters.

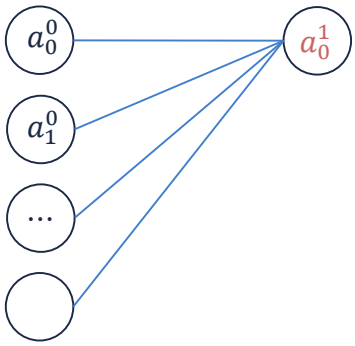


https://kim.hfg-karlsruhe.de/neural-network-chart/

28

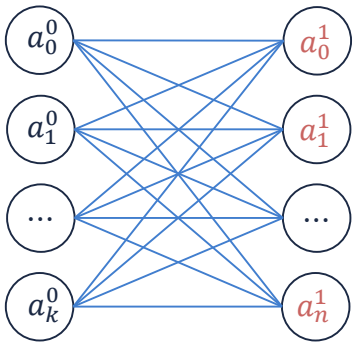
A neural network is best understood as a collection of modules or building blocks. A module takes input, applies some transformation, and returns an output. Some modules have trainable parameters, such as dense layers; others simply transform data without learning parameters, such as certain activation or pooling operations. This **modularity** is one reason neural networks are highly flexible and can be adapted to many different data types and tasks.

A neural network layer can be represented as a matrix



$$a_0^1 = \sigma(w_{0,0} \cdot a_0^0 + w_{0,1} \cdot a_1^0 + \ \dots + w_{0,n} \cdot a_n^0 + b_0)$$

A neural network layer can be represented as a matrix



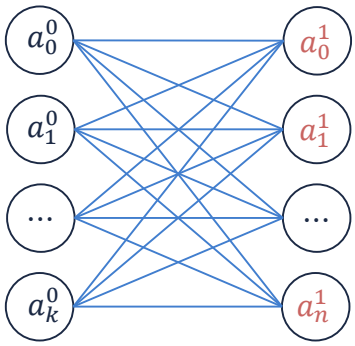
$$a_0^1 = \sigma(w_{0,0} \cdot a_0^0 + w_{0,1} \cdot a_1^0 + \ \dots + w_{0,n} \cdot a_n^0 + b_0)$$

$$a_1^1 = \sigma(w_{1,0} \cdot a_0^0 + w_{1,1} \cdot a_1^0 + \ \dots + w_{1,n} \cdot a_n^0 + b_1)$$

$$\dots$$

$$a_n^1 = \sigma(w_{k,0} \cdot a_0^0 + w_{k,1} \cdot a_1^0 + \ \dots + w_{k,n} \cdot a_n^0 + b_n)$$

A neural network layer can be represented as a matrix



$$a_0^1 = \sigma(w_{0,0} \cdot a_0^0 + w_{0,1} \cdot a_1^0 + \ \dots + w_{0,n} \cdot a_n^0 + b_0)$$

$$a_1^1 = \sigma(w_{1,0} \cdot a_0^0 + w_{1,1} \cdot a_1^0 + \ \dots + w_{1,n} \cdot a_n^0 + b_1)$$

$$\dots$$

$$a_n^1 = \sigma(w_{k,0} \cdot a_0^0 + w_{k,1} \cdot a_1^0 + \ \dots + w_{k,n} \cdot a_n^0 + b_n)$$

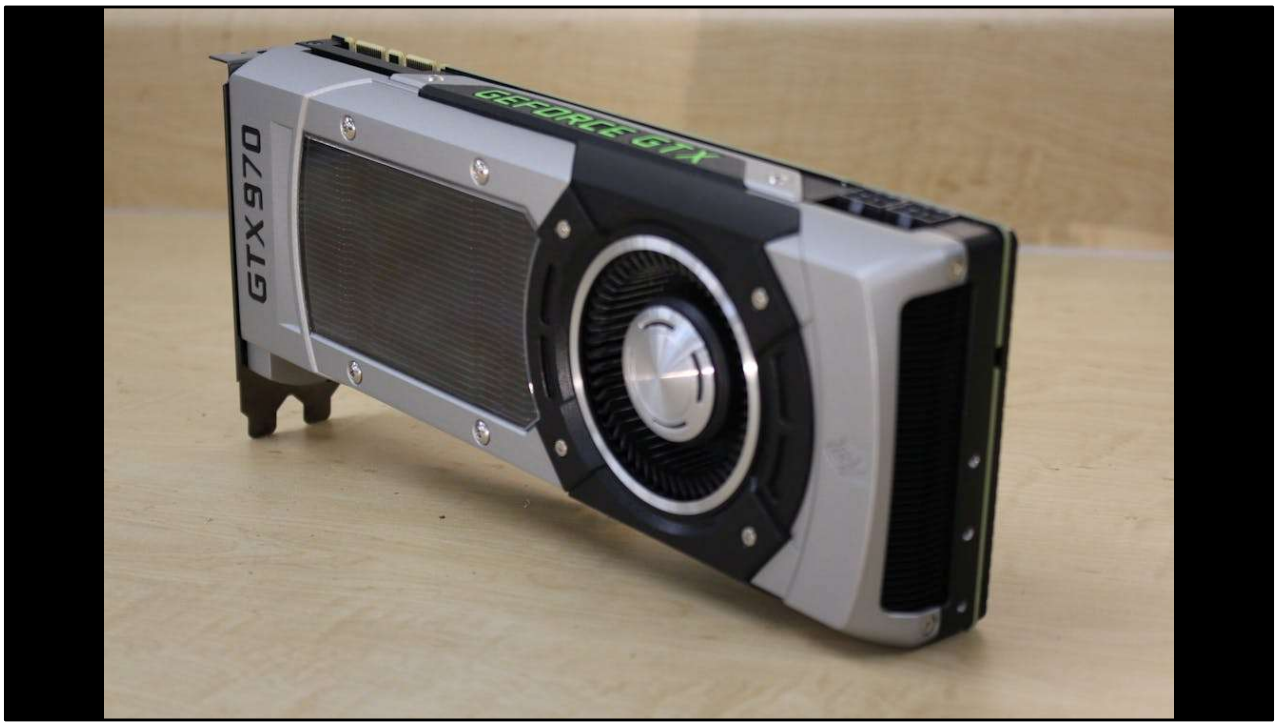
$$\downarrow$$

$$\begin{bmatrix} a_0^1 \\ a_1^1 \\ \vdots \\ a_n^1 \end{bmatrix} = \sigma\left(\begin{bmatrix} w_{0,0} & w_{0,1} & \cdots & w_{0,n} \\ w_{1,0} & w_{1,1} & \cdots & w_{1,n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{k,0} & w_{k,1} & \cdots & w_{k,n} \end{bmatrix} \begin{bmatrix} a_0^0 \\ a_1^0 \\ \vdots \\ a_n^0 \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \\ \vdots \\ b_n \end{bmatrix} \right)$$

$$a^1 = \sigma(W^1 \cdot a^0 + b^1) \quad \longleftarrow$$

31

A full layer of a neural network can be written compactly in **matrix form**. Instead of writing one equation per neuron, all neuron computations in a layer can be grouped into a matrix multiplication plus a bias vector, followed by an activation function.



Because neural network computations rely heavily on **large matrix operations**, **GPUs** are highly effective for training them. GPUs are designed for **parallel numerical computation** and therefore accelerate both the forward and backward passes, especially in deep models with many parameters.

Recap

- Neural networks can be understood as layered stacks of logistic-regression-like units.
- They can learn non-linear patterns through activation functions.
- Deeper networks can learn more complex patterns, but require more data.
- Backpropagation allows for gradient descent across many neural network layers.
- Mini-batch gradient descent balances speed and stability.
- Neural networks can be trained over many epochs.